Diseño, implementación y validación de un algoritmo para la integración numérica de Sistemas Dinámicos No Suaves tipo Filippov

Abstract

Resumen corto de lo que se hizo en el artículo.

1 Introducción

En la introducción ponemos:

- Importancia de los métodos numéricos en la solución de ecuaciones diferenciales ordinarias y la aplicación de la matemática para resolver problemas de la vida cotidiana. Resaltar lo siguiente:
  - Brecha Científica (Research Gap): Los integradores numéricos estándar de paso fijo o adaptativo (como Runge-Kutta 4/5 o LSODA) sufren de pérdida de orden de convergencia, oscilaciones espurias (chattering) o fallos de convergencia cuando las trayectorias cruzan o penetran Σ.
  - Aporte: Presentación de un algoritmo robusto y liviano desarrollado en Python que combina detección precisa de eventos (event-finding) con la convexificación de Filippov para la dinámica en la región de deslizamiento (sliding set).
- Estado del arte sobre la temática a abarcar (solución numérica de ecuaciones diferenciales ordinarias, sistemas de Filippov, Arquitectura de software)
- Como esta distribuido el artículo [1].

2 Marco referencial

Se dividirá en dos secciones:

2.1 Sistemas de Filippov

Aquí se escribe las definiciones matemáticas de los Sistemas de Filippov y la teoría sobre los métodos numéricos para ecuaciones diferenciales ordinarias.

2.2 Arquitectura de software

La arquitectura de software describe la estructura de un sistema: los componentes que lo conforman, sus interrelaciones y los principios y guías que gobiernan su diseño y evolución a lo largo del tiempo [2]. De manera complementaria, el estándar ISO/IEC/IEEE 42010 la define como los conceptos y propiedades fundamentales de un sistema en su entorno, materializados en sus elementos, sus relaciones y los principios que rigen su diseño [3]. Una buena arquitectura no es un fin en sí misma, sino un medio para alcanzar los atributos de calidad del producto, tales como disponibilidad, interoperabilidad, mantenibilidad, rendimiento, seguridad, capacidad de prueba y usabilidad [4].

Los *patrones arquitectónicos* (denominados *estilos* en la terminología de Garlan y Shaw [2]) son soluciones reutilizables a problemas de organización estructural recurrentes. Entre los estilos clásicos se encuentran el de tubos y filtros, el basado en datos, el orientado a objetos y el de **capas** (*layered* o *multilayer*) [5]. En el patrón de capas, el sistema se descompone jerárquicamente en niveles, cada uno con una responsabilidad bien delimitada, de modo que una capa únicamente puede utilizar los servicios ofrecidos por las capas inferiores [5]. Este patrón es especialmente atractivo en sistemas científicos porque favorece la separación de la configuración, el motor de cómputo, la lógica de control y la presentación de resultados; su principal costo es la posible pérdida de eficiencia por la comunicación vertical entre niveles, que en aplicaciones numéricas puede mitigarse agrupando las operaciones computacionalmente intensivas en una única capa [5].

La organización por capas es una manifestación concreta del principio de *separación de intereses* (*separation of concerns*) y del principio de *responsabilidad única*: cada módulo debe tener una única razón para cambiar [6]. En el ámbito del software científico, estas prácticas se traducen en recomendaciones concretas: escribir módulos pequeños y cohesivos, encapsular cada etapa del flujo de trabajo y evitar la duplicación de lógica, lo que incrementa la reproducibilidad y la confiabilidad de los resultados [7].

En el desarrollo del algoritmo de este trabajo se adoptó una **arquitectura modular por capas de responsabilidad única**, cuya base de configuración alimenta al motor numérico y a las capas de análisis que lo rodean. El núcleo del sistema es un motor *basado en eventos* (*event-driven*) que orquesta la integración numérica de un sistema de Filippov [8]: detecta los cruces de la superficie de conmutación, clasifica el tipo de encuentro (cruce, deslizamiento o tangencia) y comuta dinámicamente entre la integración de los campos suaves y la dinámica de deslizamiento. Esta organización, cuyos detalles se exponen en la Sección 3, surge de la teoría anterior: las capas garantizan la mantenibilidad y capacidad de pruebas de cada componente [4], [5], mientras que el patrón event-driven atiende el requisito numérico de resolver con precisión los eventos no suaves [8].

3 Arquitectura del algoritmo

Esta sección describe el flujo lógico del algoritmo implementado en Python, siguiendo la arquitectura modular por capas presentada en la Sección 2.2 y el método event-driven de Piiroinen y Kuznetsov [8]. El sistema estudiado es un modelo epidemiológico SI con control discontinuo: la población susceptible $x = S$ y la población infectada $y = I$ evolucionan según un campo vectorial f_1 (sin control) por debajo del umbral w y un campo f_2 (con control u) por encima de él. La superficie de conmutación es la recta $\Sigma = \{\mathbf{x} : h(\mathbf{x}) = 0\}$ con $h(\mathbf{x}) = y - w$, y el gradiente $\nabla h = (0, 1)$ induce la proyección de las componentes verticales de los campos.

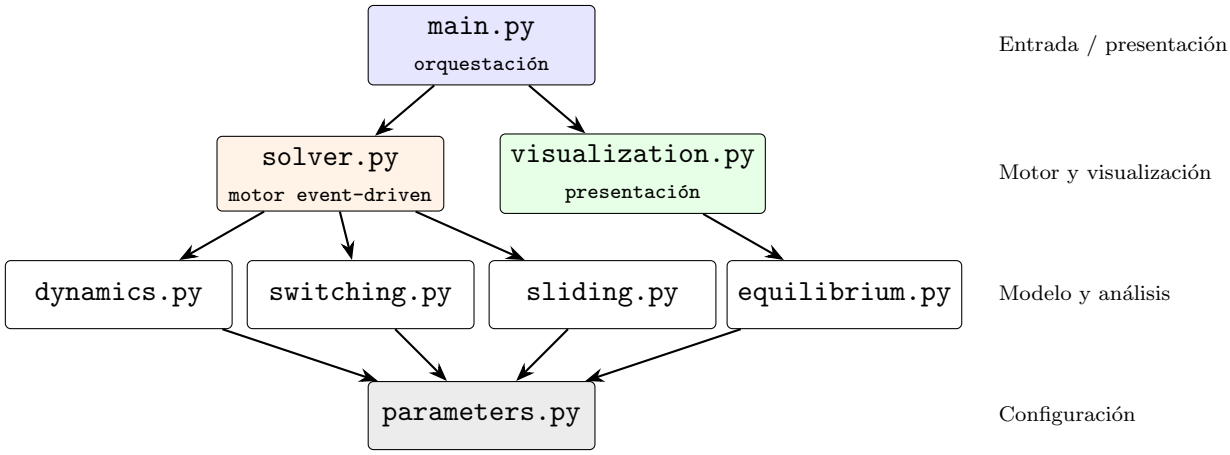


Figura 1: Arquitectura modular por capas del algoritmo. Las dependencias fluyen hacia abajo desde el punto de entrada hasta la capa de configuración.

3.1 Flujo general del bucle event-driven

El corazón del algoritmo es la función `solver.simulate()`, cuya lógica reproduce el Algoritmo 1 de [8]. El pseudocódigo completo se presenta en la Sección 3.8 y el diagrama de flujo en la Figura 2. El bucle principal se repite hasta alcanzar `max_switches` conmutaciones:

1. Determinar la región actual ($y < w$: sistema 1; $y > w$: sistema 2; $y = w$: frontera).
2. Integrar el campo vectorial correspondiente con `scipy.integrate.solve_ivp` hasta que el evento $y(t) - w = 0$ (terminal) detenga la integración.
3. Si no hubo evento, la trayectoria permaneció en su región y la simulación termina. En caso contrario, el último punto del arreglo de la solución es el punto de conmutación.
4. Clasificar el punto de cruce mediante el criterio $L = f_{1y} \cdot f_{2y}$ y actuar según el tipo (cruce, deslizamiento o tangencia).
5. Verificar que la trayectoria permanezca dentro del dominio $[a_x, b_x] \times [a_y, b_y]$; si sale, terminar.

Cada tramo integrado se almacena en una estructura `TrajectorySegment` (coordenadas, tipo de segmento) y la lista completa de segmentos, junto con los puntos de tangencia alcanzados y el punto final, se agrupa en un `SimulationResult`. De esta forma la simulación queda descompuesta en piezas reutilizables y trazables, en línea con los atributos de mantenibilidad discutidos en la Sección 2.2.

3.2 Capa de configuración: `parameters.py`

La base de la arquitectura (Figura 1) centraliza toda la configuración en tres *dataclasses* inmutables (`frozen=True`): `FilippovParams` define los parámetros del modelo (R_0, μ, θ, u, w); `DomainParams` fija los límites del plano de fase $[0, 1] \times [0, 1]$; y `SolverParams` reúne las opciones del integrador: `tiempo_max`=50, `max_switches`=8, `max_step`=0.01, tolerancias `rtol`=10^{-3} y `atol`=10^{-6}, la perturbación `eps`=10^{-6} y la tolerancia `tol`=10^{-8}. La inmutabilidad impide cambios accidentales y la inyección tipada de estos objetos como argumentos elimina las variables globales dispersas.

3.3 Modelado de la dinámica: `dynamics.py`

La capa de modelado define los dos campos vectoriales y el campo de deslizamiento. Al identificarse $\dot{x}$ con la población susceptible, se tiene `vector_field_1` (sistema 1, sin control):

$$f_1(\mathbf{x}) = \begin{pmatrix} \mu(1-x) - (\mu+\theta)R_0\, x\, y \\ (\mu+\theta)\, y\, (R_0 x - 1) \end{pmatrix}, \tag{1}$$

y `vector_field_2` (sistema 2, con control u):

$$f_2(\mathbf{x}) = \begin{pmatrix} \mu(1-x) - (\mu+\theta)R_0\, x\, y \\ (\mu+\theta)\, y\, (R_0 x - 1) - u \end{pmatrix}. \tag{2}$$

El punto clave es que **la componente x es idéntica en ambos sistemas** ($f_{1x} \equiv f_{2x}$); solo difiere la componente vertical por el término $-u$. Por ello, la dinámica de deslizamiento sobre Σ ($y = w$) se reduce a la ODE unidimensional `sliding_field_1d`:

$$\dot{x} = \mu(1-x) - (\mu+\theta)R_0\, x\, w. \tag{3}$$

3.4 Análisis geométrico del plano de fase

Los módulos `equilibrium.py` y `switching.py` precalculan la información geométrica que orienta la simulación.

Puntos de equilibrio. Para el sistema 1, el equilibrio endémico es $(S^*, I^*) = (1/R_0,\ \mu(R_0-1)/((\mu+\theta)R_0))$. Para el sistema 2, la condición $\dot{y} = 0$ conduce a la ecuación cuadrática $\mu R_0 x^2 - (\mu + \mu R_0 - uR_0)x + \mu = 0$; si su discriminante es negativo no existen equilibrios reales y `compute_equilibria` devuelve `NaN`.

Puntos tangentes. Un campo es tangente a Σ cuando su proyección vertical se anula ($f_{iy} = 0$):

$$T_1 = \left(\frac{1}{R_0}, w\right), \qquad T_2 = \left(\frac{1}{R_0}\left(1 + \frac{u}{(\mu+\theta)\, w}\right), w\right). \tag{4}$$

Entre T_1 y T_2 se encuentra la zona de deslizamiento, donde los campos apuntan en direcciones opuestas.

Criterio de conmutación. Con $h = y - w$, la proyección de cada campo sobre ∇h es su componente vertical, por lo que el producto $L = f_{1y}(x, w) \cdot f_{2y}(x, w)$ clasifica el comportamiento en Σ [8]: si $L > 0$ ambos campos apuntan al mismo lado (`CROSSING`); si $L < 0$ apuntan en direcciones opuestas

(`SLIDING`); si $L \approx 0$ al menos un campo es tangente (`TANGENCY`). La enumeración `SwitchType` y la *dataclass* `SwitchAnalysis` (L, f_{1y}, f_{2y}, tipo) encapsulan esta clasificación tipo-segura.

Formulación clásica de Filippov. El deslizamiento admite una descripción equivalente mediante la combinación convexa que anula la proyección vertical sobre Σ:

$$f_F(\mathbf{x}) = (1-\beta)\, f_1(\mathbf{x}) + \beta\, f_2(\mathbf{x}), \qquad \beta = \frac{f_{1y}}{f_{1y} - f_{2y}}, \tag{5}$$

denominada convexificación de Filippov [9]. Como $f_{1x} \equiv f_{2x}$, la componente horizontal de f_F no depende de β y coincide con la ODE unidimensional (3); por ello `sliding_field_1d` implementa exactamente el campo de Filippov restringido a Σ. La condición de deslizamiento $\beta \in [0,1]$ es equivalente al criterio $L < 0$: $\beta \in (0,1)$ se cumple si y solo si f_{1y} y f_{2y} tienen signos opuestos (es decir $L < 0$), y los extremos $\beta = 0$ y $\beta = 1$ corresponden a las tangencias en T_1 y T_2 ($L = 0$).

3.5 Integración numérica y detección de cruces

La capa `solver.py` construye, mediante la fábrica `make_crossing_event`, la función de evento $g(t) = y(t) - w$ con `terminal=True` y `direction`=+1 (si parte de la región inferior) o -1 (si parte de la superior). `solve_ivp` detecta el cambio de signo de $g(t)$ entre pasos consecutivos y localiza el instante de impacto t^* con $g(t^*) = 0$ resolviendo la raíz dentro del intervalo de integración: el refinamiento de scipy interpola el polinomio asociado al esquema de Runge–Kutta interno, lo que es conceptualmente equivalente a un método de búsqueda de raíces (bisección/Brent) con la ventaja de ser consistente con el orden del integrador. De este modo, el último punto de la solución coincide con la intersección con Σ, $x(t^*)$, salvo la tolerancia del integrador. La trayectoria se mantiene suficientemente fina mediante `max_step`=0.01 para evitar saltar la frontera entre pasos.

Cuando `compute_switching_analysis` clasifica el cruce:

- **CROSSING**: se empuja la trayectoria al otro lado con la perturbación infinitesimal $y \leftarrow w \pm \varepsilon$, $\varepsilon = 10^{-6}$, garantizando que el siguiente paso parta de la región correcta.
- **SLIDING**: se invoca la integración de deslizamiento de la Sección 3.6.
- **TANGENCY**: se registra el punto en `tangent_hits` y se decide el lado de salida según el signo de la componente vertical del campo no tangente.

Si la trayectoria cae *exactamente* sobre la frontera al inicio de una iteración, se aplica el mismo procedimiento de clasificación y posteriormente se ejecuta `continue`, evitando reintentar la integración desde $y = w$ con un subsistema indeterminado.

3.6 Dinámica de deslizamiento: `sliding.py`

Bajo la clasificación `SLIDING`, la función `integrate_sliding` integra la ODE unidimensional (3) sobre Σ con dos eventos terminales adicionales que detectan la llegada a T_1 (evento $x - x_{T_1} = 0$) y a T_2 (evento $x - x_{T_2} = 0$), con dirección 0 (cualquier sentido de cruce). Los indicadores `reached_T1` y `reached_T2` se leen de `sol.t_events[0]` y `sol.t_events[1]`, respectivamente, y la coordenada final del deslizamiento se toma de `sol.y_events`. Al salir, se evalúa de nuevo f_{1y}: si es positiva la trayectoria escapa hacia arriba ($y = w + \varepsilon$) y si es negativa hacia abajo ($y = w - \varepsilon$). Este mecanismo implementa la salida del deslizamiento en los puntos tangentes, donde el parámetro de convexidad β alcanza los extremos del intervalo $[0, 1]$ ($\beta \notin (0, 1)$) [8].

3.7 Orquestación y visualización

`main.py` actúa como punto de entrada: instancia las tres *dataclasses* de configuración, define las condiciones iniciales, ejecuta `simulate()` para cada una y delega el dibujo en `visualization.plot_phase_portrait()`. La capa de visualización evalúa los campos vectoriales sobre la cuadrícula de forma **vectorizada con** `numpy` (broadcasting matricial en lugar de bucles anidados, aproximadamente un orden de magnitud más rápido), dibuja la frontera con tres estilos (punteada fuera de $[T_1, T_2]$ y continua en la zona de deslizamiento) y superpone segmentos de trayectoria por tipo, equilibrios, puntos tangentes y condiciones iniciales.

3.8 Detalles computacionales y pseudocódigo

Desde el punto de vista computacional, el algoritmo alterna integraciones suaves de alto orden (`solve_ivp` con control de tolerancia `rtol`/`atol`) con la resolución de raíces de eventos. La detección de eventos evita el *chattering* de los métodos de paso fijo y mantiene el número de evaluaciones de función acotado.

En cuanto al uso de las estructuras de datos, todo el cálculo descansa en arreglos `numpy`: el integrador representa el estado como `V = [x, y]` y los campos vectoriales de `dynamics.py` reciben ese arreglo y lo procesan con operaciones vectoriales (`np.sqrt`, diferencias, productos entre escalares del mismo tipo), de modo que no hay conversiones intermedias tipo Python–NumPy ni bucles explícitos en el punto caliente del cómputo; la única conversión a listas ocurre al momento de *almacenar* los segmentos ya integrados en `TrajectorySegment`. Esta elección evita el costo de interpretación de Python en cada evaluación de función y aprovecha el *unboxing* que `solve_ivp` utiliza internamente en sus etapas de Runge–Kutta. En la capa de visualización, además, los campos se evalúan sobre la cuadrícula completa con *broadcasting* de `numpy` (operaciones matriciales en lugar de bucles anidados),

aproximadamente un orden de magnitud más rápido que la evaluación punto a punto.

La integración con `scipy.integrate.solve_ivp` aporta dos ventajas numéricas centrales al esquema event-driven. Primero, el paso de integración es *adaptativo*: el tamaño del paso se ajusta en cada etapa para satisfacer el criterio $|\text{error}| \leq atol + rtol\,|y|$, concentrando evaluaciones donde la solución varía rápido y alargando los pasos en las regiones suaves, lo que equilibra precisión y costo. Segundo, la localización del evento $g(t) = y(t) - w$ no depende del refinamiento del paso sino de la búsqueda de raíz del polinomio interpolante del esquema interno, lo que produce el instante de impacto t^* con precisión consistente con el orden del integrador y mantiene el número de llamadas a la función acotado e independiente de la suavidad de la frontera. El pseudocódigo del bucle completo se muestra en el Algoritmo 1 y el diagrama de flujo en la Figura 2 (los detalles numéricos de costo y precisión se evalúan en la Sección 4).

Algoritmo 1 Bucle event-driven de Filippov (adaptado de [8], Algoritmo 1)

Entrada: (x_0, y_0), fp, dp, sp
$x, y \leftarrow x_0, y_0$; $n \leftarrow 0$; $result \leftarrow \text{SimulationResult}(x_0, y_0)$
while $n < sp.\text{max_switches}$ **do**
  if $y < w$ **then**
    $vf \leftarrow f_1$; $tipo \leftarrow$ sistema1
  else if $y > w$ **then**
    $vf \leftarrow f_2$; $tipo \leftarrow$ sistema2
  else
    $a \leftarrow \text{compute_switching_analysis}(x, w)$ ▷ criterio $L = f_{1y}\, f_{2y}$
    if $a =$ SLIDING **then**
      $slide \leftarrow \text{integrate_sliding}(x, w, T_1, T_2)$
      $x \leftarrow slide.x_{\text{fin}}$; $y \leftarrow w + \varepsilon$ si $f_{1y} > 0$, si no $w - \varepsilon$
    else if $a =$ TANGENCY **then**
      registrar tangencia; $y \leftarrow w \pm \varepsilon$ (signo del campo no tangente)
    else
      $y \leftarrow w \pm \varepsilon$ ▷ cruce en la frontera
    end if
    continue
  end if
  $event \leftarrow$ cruce terminal $y(t) - w = 0$, dirección ± 1
  $sol \leftarrow \text{solve_ivp}(vf, [0, t_{\max}], [x, y], \text{events} = event)$
  guardar segmento $(sol.y, tipo)$
  if no hubo evento **then**
    break
  end if
  $(x, y) \leftarrow$ último punto de sol ▷ punto de cruce en Σ
  $a \leftarrow \text{compute_switching_analysis}(x, w)$
  if $a =$ CROSSING **then**
    $y \leftarrow w + \varepsilon$ si $tipo =$ sistema1, si no $y \leftarrow w - \varepsilon$
  else if $a =$ SLIDING **then**
    $slide \leftarrow \text{integrate_sliding}(x, w, T_1, T_2)$
    guardar segmento sliding; $x \leftarrow slide.x_{\text{fin}}$
    $y \leftarrow w + \varepsilon$ si $f_{1y} > 0$, si no $w - \varepsilon$
  else
    registrar tangencia; $y \leftarrow w \pm \varepsilon$ (signo del otro campo)
  end if
  $n \leftarrow n + 1$
  if (x, y) fuera de dp **then**
    break
  end if
end while

$result.\text{final_point} \leftarrow (x, y)$; **return** $result$

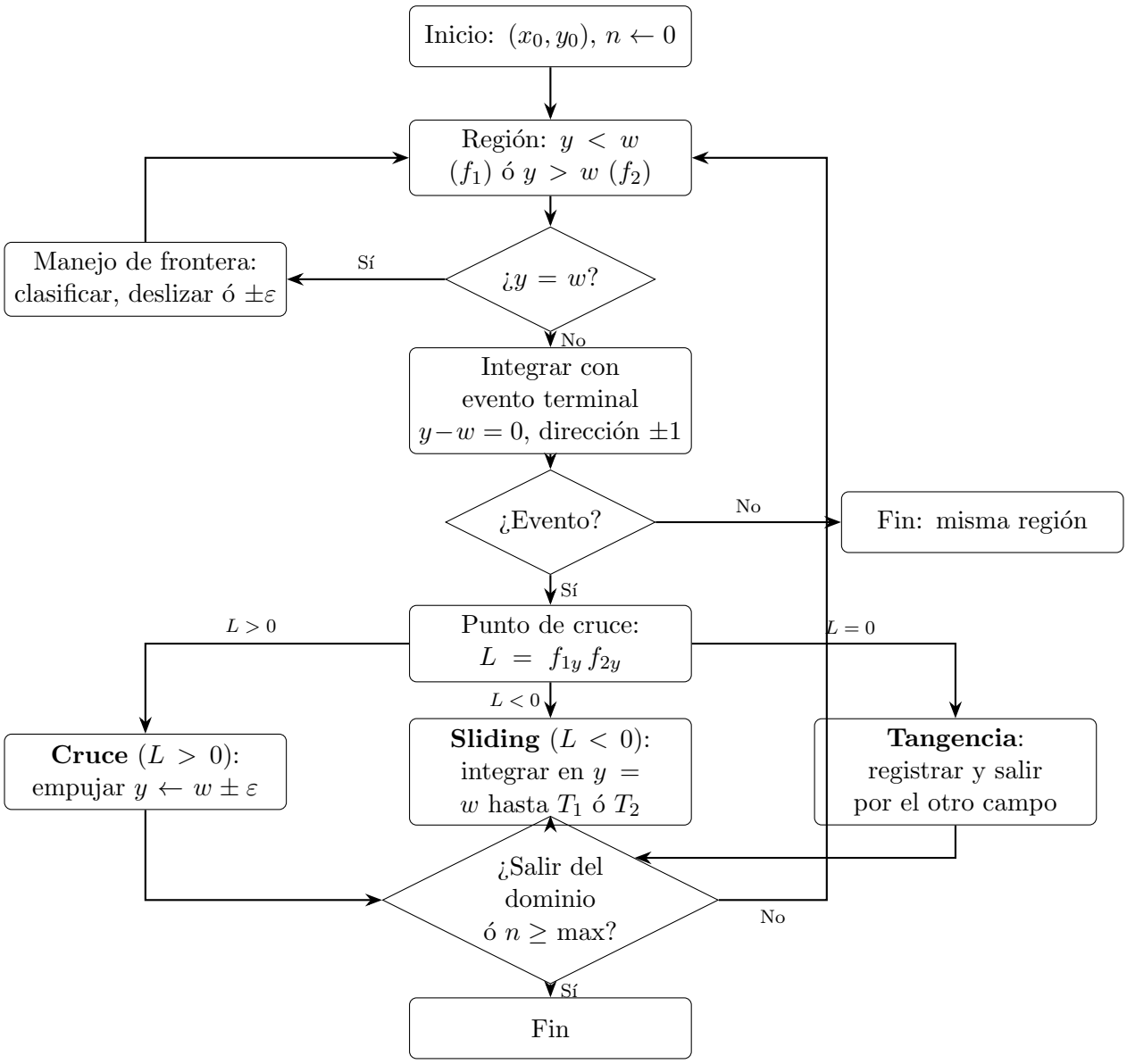


Figura 2: Diagrama de flujo del bucle event-driven implementado en `solver.simulate()`.

4 Casos de estudio y resultados numéricos

Para presentar la implementación del algoritmo en los sistemas de Filippov, se presentarán 2 casos de estudio: un sistema en $\mathbb{R}^2$ y otro sistema en $\mathbb{R}^3$.

4.1 Caso 1

Se presenta el planteamiento y solución numérica del modelo Lokta-Volterra en $\mathbb{R}^2$ planteado en [10].

4.2 Caso 2

Presentación del planteamiento y análisis de un modelo tridimensional tipo Filippov.

4.3 Métricas de evaluación del algoritmo

- Error cuadrático medio respecto a soluciones analíticas/de alta precisión.
- Tiempo de ejecución (CPU time) frente a solvers nativos de Python.
- Control del número de evaluaciones de función (N_{eval}).

5 Conclusiones y Discusión

Hablar sobre los resultados obtenidos y tropiesos que se tienen para llegar a las soluciones o mejorar el rendimiento del código, resaltando:

- Ventajas operativas frente a métodos convencionales sin detección de eventos.
- Complejidad computacional e impacto en el desarrollo de software académico.
- Limitaciones del algoritmo (ej. superficies de conmutación múltiples o no diferenciables).
- Se tiene una dependencia de un estudio analítico previo.

Referencias

[1] A. Wang, Y. Gong, D. Bai, W. Zhou, and S. R. Smith, "A Filippov model describing the effect of social distancing in controlling infectious diseases," *Bulletin of Mathematical Biology*, vol. 87, no. 11, p. 156, 2025. DOI: 10.1007/s11538-025-01530-1

[2] D. Garlan and M. Shaw, "An introduction to software architecture," Carnegie Mellon University, Pittsburgh, PA, Technical Report CMU-CS-94-166, 1994.

[3] ISO/IEC/IEEE, *Systems and software engineering — architecture description*, International Standard ISO/IEC/IEEE 42010:2011, 2011.

[4] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Boston, MA: Addison-Wesley Professional, 2021.

[5] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Chichester, UK: John Wiley & Sons, 1996.

[6] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ: Prentice Hall, 2003.

[7] G. Wilson et al., "Best practices for scientific computing," *PLoS Biology*, vol. 12, no. 1, e1001745, 2014. DOI: 10.1371/journal.pbio.1001745

[8] P. T. Piiroinen and Y. A. Kuznetsov, "An event-driven method to simulate Filippov systems with accurate computing of sliding motions," *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–24, 2008. DOI: 10.1145/1356052.1356054

[9] A. F. Filippov, *Differential Equations with Discontinuous Righthand Sides* (Mathematics and its Applications). Kluwer Academic Publishers, 1988, vol. 18. DOI: 10.1007/978-94-015-7793-9

[10] Y. A. Kuznetsov, S. Rinaldi, and A. Gragnani, "One-parameter bifurcations in planar Filippov systems," *International Journal of Bifurcation and Chaos*, vol. 13, no. 8, pp. 2157–2188, 2003. DOI: 10.1142/S0218127403007874